Table of Contents

Introduction *1*
Getting Started 1
Props 2
Destructured Props Format 6
Simple Alert Components 8
Modal Dialogs 11
Navigation 14
Tabs, Parent Detail, goBack() 14
List-Detail View Navigation 19
Conclusion *22*

Introduction

This document introduces Props, Alerts and Model dialogs in React Native. It also provides review for implementing tabs and navigation between parent and child views but within a Typescript environment.

Getting Started

Example 1: Creating a Typescript Project

To help with examples that follow, this example contains steps for setting up a typescript React Native app. Run this command to create a basic React Native app with TypeScript using Expo:

```
npx create-expo-app myapp --template blank-typescript
cd myapp
```

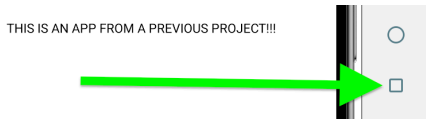
> **Mac users who may have permissions issues**
> Remember to use **sudo** if you encounter permission issues. Also, run this command at the root of your project to grant read write permission on my project folder and sub folders:
> `sudo chown -R $USER ./`

To run the app, use this command:

```
npx expo start
```

Be sure to press **a** to run the app in the Android emulator which should already be running.

> **Tip:** I often find the old app stays on the screen, so I go to the home screen and swipe it away to fully close it. After that, I perform a hard refresh to reload the app cleanly.

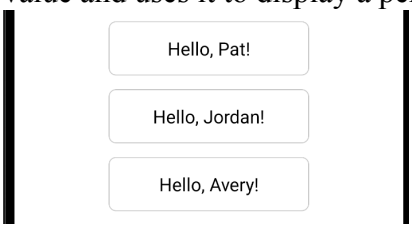


Props

Props are values passed from a parent component to a child component. The props definition is stored in a TypeScript file.

Example 2: Props Introduction

This example introduces props by creating a reusable `Greeting` component that receives a `name` value and uses it to display a personalized message.



The `GreetingTypes.ts` file defines a custom type that specifies the structure of the props, including the `name` field.

GreetingProps.ts

```
export type GreetingProps = {
  name: string;
};
```

React Native components written in TypeScript are stored in .tsx files. This component is defined as a function that receives props, which is typed using GreetingProps.

Greeting.tsx

```
import { View, Text, StyleSheet } from 'react-native';
import { GreetingProps } from './GreetingProps';

export default function Greeting(props: GreetingProps) {
  return (
    <View style={styles.box}>
      <Text style={styles.text}>Hello, {props.name}!</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  box: {
    marginBottom: 15,
    padding: 15,
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    width: 220,
    alignItems: 'center',
  },
  text: {
    fontSize: 20,
  },
});
```

The app renders multiple Greeting components, each with a different name passed through props.

App.tsx

```
import { StatusBar }        from 'expo-status-bar';
import { StyleSheet, View } from 'react-native';
import Greeting             from './Greeting';

export default function App() {
  return (
    <View style={styles.container}>
      <Greeting name="Pat" />
      <Greeting name="Jordan" />
      <Greeting name="Avery" />
      <StatusBar style="auto" />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Exercise 1 (2 marks)

Modify the example so that the `GreetingProps` type includes three fields: `firstName`, `lastName`, and `id` (number). Update the `Greeting` component to display all three values. Finally, update the `App` component to pass the correct `firstName`, `lastName`, and `id` props to each `Greeting` component.

Show your revised **GreetingProps.ts** code as text with proper naming here (no screenshots please):

```
export type GreetingProps = {
  firstName: string;
  lastName: string;
  id: number;
  };
```

Show your revised **Greeting.tsx** code as text with proper naming here (no screenshots please):

```
import { View, Text, StyleSheet } from 'react-native';
import { GreetingProps } from './GreetingProps';

export default function Greeting(props: GreetingProps) {
  return (
    <View style={styles.box}>
      <Text style={styles.text}>Hello, {props.firstName} {props.lastName} (ID: {props.id})!</
Text>
    </View>
  );
}
const styles = StyleSheet.create({
  box: {
    marginBottom: 15,
    padding: 15,
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    width: 220,
    alignItems: 'center',
  },
  text: {
    fontSize: 20,
  },
});
```

Show your revised **App.tsx** code as text with proper naming here (no screenshots please):

```
import { StatusBar }         from 'expo-status-bar';
import { StyleSheet, View } from 'react-native';
import Greeting              from './Greeting';

export default function App() {
  return (
    <View style={styles.container}>
      <Greeting firstName="Hali" lastName="Imanpanah" id={1} />
<Greeting firstName="Jordan" lastName="Lee" id={2} />
<Greeting firstName="Avery" lastName="Brown" id={3} />
      <StatusBar style="auto" />
    </View>
  );
```

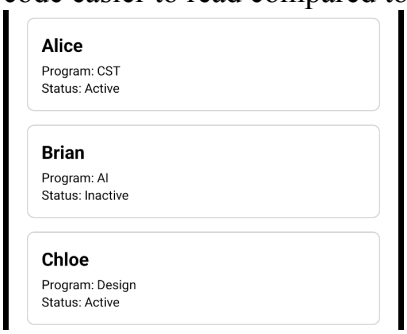
```
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Destructured Props Format

The destructured props style displays each individual property in the component function header.

Example 3: Destructured Props

This example demonstrates props using a **destructured props format**, where the properties (`name`, `program`, `isActive`) are displayed directly in the function parameters. This makes the code easier to read compared to accessing values through the props object.



For simplicity all code is located in **App.tsx** but normally the `Student` component would be in a separate tsx file.

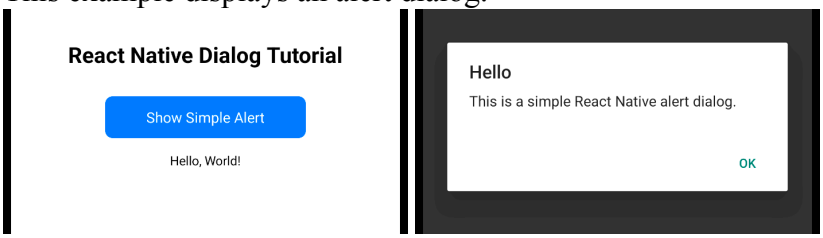
```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View } from 'react-native';

type StudentProps = {
  name: string;
  program: string;
  isActive: boolean;
};

function Student({ name, program, isActive }: StudentProps) {
  return (
    <View style={styles.card}>
      <Text style={styles.title}>{name}</Text>
      <Text>Program: {program}</Text>
      <Text>Status: {isActive ? 'Active' : 'Inactive'}</Text>
    </View>
  );
}

export default function App() {
  return (
    <View style={styles.container}>
      <Student name="Alice" program="CST" isActive={true} />
      <Student name="Brian" program="AI" isActive={false} />
      <Student name="Chloe" program="Design" isActive={true} />
      <StatusBar style="auto" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
  },
  card: {
    width: '100%',
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 15,
    marginBottom: 15,
  },
  title: {
    fontSize: 20,
```

Simple Alert Components

The Alert component in React Native allows you to pass a title (string), message (string) and an array of buttons. With the simple alert components, you cannot add custom components like icons or images and you cannot change its layout or styling.

Example 4: Simple Alert Dialogs

This example displays an alert dialog.



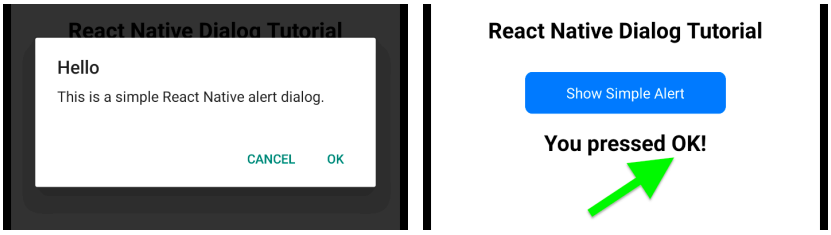
App.tsx

```
import React, { useState } from 'react';
import { Alert,  Pressable, StyleSheet, Text, View } from 'react-native';

export default function App() {
  const [someText, setText] = useState("Hello, World!");

  const showSimpleAlert = () => {
    // alert() only receives title, subtitle and buttons.
    Alert.alert(
      'Hello',
      'This is a simple React Native alert dialog.',
      [
        {
          text: 'OK',
          onPress: () => {Alert.alert('Result', 'You pressed okay!')},
        },
      ]
    );
  };
  return (
    <View style={styles.container}>
      <Text style={styles.title}>React Native Dialog Tutorial</Text>

      <Pressable style={styles.button} onPress={showSimpleAlert}>
        <Text style={styles.buttonText}>Show Simple Alert</Text>
      </Pressable>
      <Text>{someText}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
  },
  title: {
    fontSize: 24,
    marginBottom: 30,
    fontWeight: 'bold',
    textAlign: 'center',
  },
  button: {
    backgroundColor: '#007AFF',
    paddingVertical: 12,
```

Exercise 2 (2 marks)

Modify the Example 4 code so the dialog displays two buttons as shown at the left in Figure 1. Also change the text that is displayed as shown at the right of Figure 1.

Figure 1: Simple dialog (left). Main screen shows button selected (right).



Show all of your revised app.tsx file as text here with proper naming. Show your text with the same styling. No screenshots please.

```
import React, { useState } from 'react';
import { Alert,  Pressable, StyleSheet, Text, View } from 'react-native';

export default function App() {
  const [someText, setText] = useState("Hello, World!");

  const showSimpleAlert = () => {
    // alert() only receives title, subtitle and buttons.
    Alert.alert(
      'Hello',
      'This is a simple React Native alert dialog.',
      [
        {
          text: 'Cancel',
          onPress: () => setText('You pressed Cancel!'),
        },
        {
          text: 'OK',
          onPress: () => setText('You pressed OK!'),
        },
      ]
    );
  };
  return (
    <View style={styles.container}>
      <Text style={styles.title}>React Native Dialog Tutorial</Text>

      <Pressable style={styles.button} onPress={showSimpleAlert}>
        <Text style={styles.buttonText}>Show Simple Alert</Text>
      </Pressable>
      <Text>{someText}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
```

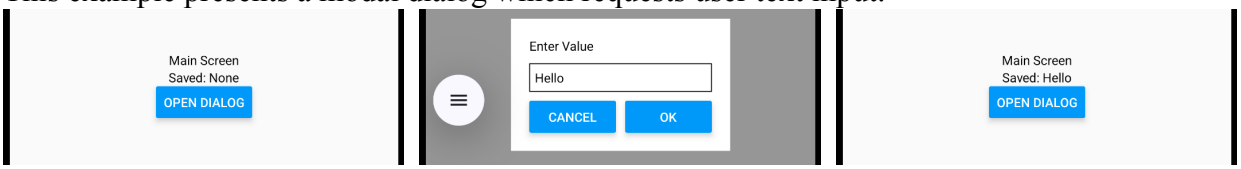
```
    padding: 20,
  },
  title: {
    fontSize: 24,
    marginBottom: 30,
    fontWeight: 'bold',
    textAlign: 'center',
  },
  button: {
    backgroundColor: '#007AFF',
    paddingVertical: 12,
    paddingHorizontal: 20,
    borderRadius: 8,
    marginBottom: 15,
    minWidth: 220,
  },
  buttonText: {
    color: 'white',
    textAlign: 'center',
    fontSize: 16,
  },
});
```

Modal Dialogs

Modal dialogs appear in front of the screen to require a user response. Unlike alerts, they are fully customizable and allow you to design your own layout, inputs, and buttons.

Example 5: Modal Dialogs

This example presents a modal dialog which requests user text input.

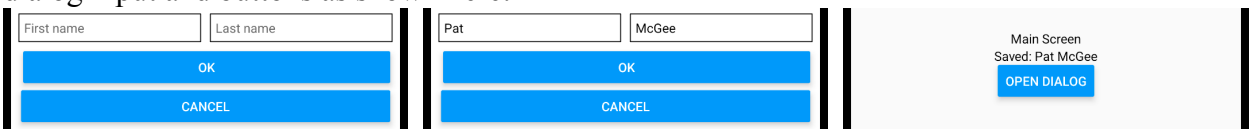


App.tsx

```
import React, { useState } from 'react';
import { View, Text, Modal, TextInput, Button } from 'react-native';

export default function App() {
  const [visible, setVisible] = useState(false);
  const [input, setInput] = useState('');
  const [saved, setSaved] = useState('');

  const handleOk = () => {
    setSaved(input);
    setVisible(false);
    setInput('');
  };

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center',
                   backgroundColor: 'rgba(0,0,0,0.4)' }}>
      <Text>Main Screen</Text>
      <Text>Saved: {saved || 'None'}</Text>
      <Button title="Open Dialog" onPress={() => setVisible(true)} />

      <Modal visible={visible} transparent>
        <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
          <View style={{ backgroundColor: 'white', padding: 20 }}>

            <Text>Enter Value</Text>

            <TextInput
              placeholder="Type here"
              value={input}
              onChangeText={setInput}
              style={{ borderWidth: 1, marginVertical: 10, padding: 5, width: 200 }}
            />

            {/* BUTTON ROW */}
            <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
              <View style={{ flex: 1, marginRight: 5 }}>
                <Button title="Cancel" onPress={() => setVisible(false)} />
              </View>

              <View style={{ flex: 1, marginLeft: 5 }}>
                <Button title="OK" onPress={handleOk} />
              </View>
            </View>

          </View>
        </View>
      </Modal>
```

Exercise 3 (4 marks)

Modify the code in Example 5 to include a second text box in React Native. Adjust the modal dialog input and buttons as shown here:



Show all of your code for App.tsx as text with proper naming here. No screenshots please.

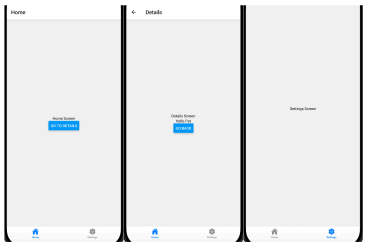
```
import React, { useState } from 'react';
import { View, Text, Modal, TextInput, Button } from 'react-native';

export default function App() {
  const [visible, setVisible] = useState(false);
  const [input, setInput] = useState('');
  const [saved, setSaved] = useState('');
  const [input2, setInput2] = useState('');


  const handleOk = () => {
    setSaved(input + ' ' + input2);
    setVisible(false);
    setInput('');
    setInput2('');
  };

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center',
                   backgroundColor: 'rgba(0,0,0,0.4)' }}>
      <Text>Main Screen</Text>
      <Text>Saved: {saved || 'None'}</Text>
      <Button title="Open Dialog" onPress={() => setVisible(true)} />

      <Modal visible={visible} transparent>
        <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
          <View style={{ backgroundColor: 'white', padding: 20, width: 400}}>

            <View style={{ flexDirection: 'row' }}>
              <TextInput placeholder="First name" value={input} onChangeText={setInput}
              style={{ borderWidth: 1, marginVertical: 10, padding: 5, flex: 1, marginRight: 5 }} />
              <TextInput placeholder="Last name" value={input2} onChangeText={setInput2}
              style={{ borderWidth: 1, marginVertical: 10, padding: 5, flex: 1, marginLeft: 5 }} />
              </View>

            {/* BUTTON ROW */}
            <View>
            <View style={{ marginBottom: 5 }}>
              <Button title="OK" onPress={handleOk} /></View>
              <Button title="Cancel" onPress={() => setVisible(false)} />
            </View>

          </View>
        </View>
      </Modal>

    </View>
  );
}
```

Navigation

Tabs, Parent Detail, goBack()

Example 6: Enabling Navigation

This example demonstrates how to set up stack navigation with tabs in React Native and shows how to navigate from a main screen to a detail screen. Run these installs to obtain the required packages.



In the same directory as package.json, run these installs to obtain the required packages to enable the navigation and icons.

```
npx expo install react-native-screens react-native-safe-area-context
npm install @react-navigation/native-stack
npm install @react-navigation/bottom-tabs
npx expo install @expo/vector-icons
```

App.tsx

```
import React from 'react';
import { View, Text, Button } from 'react-native';

import { NavigationContainer }        from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator }   from '@react-navigation/bottom-tabs';

import { Ionicons } from '@expo/vector-icons';

// TYPES
type RootStackParamList = {
  HomeMain: undefined;
  Details: { name: string };
};

type TabParamList = {
  Home: undefined;
  Settings: undefined;
};

const Stack = createNativeStackNavigator<RootStackParamList>();
const Tab = createBottomTabNavigator<TabParamList>();

// HOME SCREEN
function HomeScreen({ navigation }: any) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Home Screen</Text>

      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details', { name: 'Pat' })}
      />
    </View>
  );
}

// DETAILS SCREEN
function DetailsScreen({ route, navigation }: any) {
  const { name } = route.params;

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Details Screen</Text>
      <Text>Hello {name}</Text>

      <Button title="Go Back" onPress={() => navigation.goBack()} />
    </View>
```

Exercise 4 (2 marks)

Add an About tab and screen to Example 6. Show all code for App.tsx as text (no screenshots please).

```
import React from 'react';
import { View, Text, Button } from 'react-native';

import { NavigationContainer }        from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { createBottomTabNavigator }   from '@react-navigation/bottom-tabs';

import { Ionicons } from '@expo/vector-icons';

// TYPES
type RootStackParamList = {
  HomeMain: undefined;
  Details: { name: string };
};

type TabParamList = {
  Home: undefined;
  Settings: undefined;
  About: undefined;
};

const Stack = createNativeStackNavigator<RootStackParamList>();
const Tab = createBottomTabNavigator<TabParamList>();

// HOME SCREEN
function HomeScreen({ navigation }: any) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Home Screen</Text>

      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details', { name: 'Hali' })}
      />
    </View>
  );
}

// DETAILS SCREEN
function DetailsScreen({ route, navigation }: any) {
  const { name } = route.params;

  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Details Screen</Text>
      <Text>Hello {name}</Text>

      <Button title="Go Back" onPress={() => navigation.goBack()} />
    </View>
  );
}

// SETTINGS SCREEN
function SettingsScreen() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Settings Screen</Text>
    </View>
  );
}

function AboutScreen() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>About Screen</Text>
    </View>
  );
}

// STACK inside HOME TAB
function HomeStack() {
  return (
```

```
    <Stack.Navigator>
      <Stack.Screen
        name="HomeMain"
        component={HomeScreen}
        options={{ title: 'Home' }}
      />
      <Stack.Screen name="Details" component={DetailsScreen} />
    </Stack.Navigator>
  );
}

// TABS with icons
function MyTabs() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        headerShown: false,

        tabBarIcon: ({ color, size }) => {
          let iconName: keyof typeof Ionicons.glyphMap;

          if (route.name === 'Home') {
            iconName = 'home';
          } else {
            iconName = 'settings';
          }

          return <Ionicons name={iconName} size={size} color={color} />;
        },
      })}
    >
      <Tab.Screen name="Home" component={HomeStack} />
      <Tab.Screen name="Settings" component={SettingsScreen} />
      <Tab.Screen name="About" component={AboutScreen} />
    </Tab.Navigator>
  );
}

// APP
export default function App() {
  return (
    <NavigationContainer>
      <MyTabs />
    </NavigationContainer>
  );
}
```

Exercise 5 (1 mark)

What does `navigation.goBack()` do?

navigates the user back to the previous screen in the navigation stack.

Exercise 6 (1 mark)

What is the difference between the tab navigator and the stack navigator in this example?

tab navigator shows tabs at the bottom to switch between screens (Home, Settings, About) but stack navigator places screens on top of each other, allowing us to go forward and back

Exercise 7 (1 mark)

Why is the `HomeStack` placed inside the `Home` tab instead of putting all screens directly in the tab navigator?

is placed inside Home tab so that Home tab can have its own navigation between 3 screens without affecting other tabs.

Exercise 8 (1 mark)

What would happen if headerShown: false was removed from the tab navigator?

We'd see two headers stacked on top of each other which one from the tab navigator and one from the stack navigator.

Example 7: Passing Parameters to a Screen

This example provides additional practice working with parameters when navigating between screens. For this example the parameters are set up to be passed to the Settings screen.

Starting with Example 6, modify your tab like this:

```
<Tab.Screen
  name="Settings"
  component={SettingsScreen}
  initialParams={{
  firstName: 'Pat',
  studentId: 'A12345',
  }} />
```

The parameter list for settings needs an adjustment:

```
type TabParamList = {
  Home: undefined;
  Settings: { firstName: string; studentId: string };
};
```

The SettingsScreen header needs an adjustment as well.

```
function SettingsScreen({ route }: any) {
    const { firstName, studentId } = route.params;
```

Exercise 9 (1 mark)

Make the changes described in Example 7. Then modify the SettingsScreen function further to show the first name and studentId on the screen. Show a screenshot of the output after making the changes.

List-Detail View Navigation

Example 8: List Detail View

This example provides practice navigating from a parent screen to a child detail screen using a list-to-detail pattern. In the same directory as package.json run these installs to enable the required navigation.

```
npx expo install @react-navigation/native
npx expo install react-native-screens react-native-safe-area-context
npx expo install @react-navigation/native-stack
```

Here are the different components.

DetailScreen.tsx

```
import { View, Text } from 'react-native';

export default function DetailScreen({ route }: any) {
  const { item } = route.params;

  return (
    <View>
      <Text style={{ fontSize: 30 }}>
        {item.name}
      </Text>
      <Text>ID: {item.id}</Text>
    </View>
  );
}
```

ListScreen.tsx

```
import { View, Text, FlatList, TouchableOpacity } from 'react-native';

const data = [
  { id: '1', name: 'Apple' },
  { id: '2', name: 'Banana' },
  { id: '3', name: 'Orange' },
];

export default function ListScreen({ navigation }: any) {
  return (
    <View>
      <FlatList
        data={data}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <TouchableOpacity
            onPress={() =>
              navigation.navigate('Detail', { item: item })
            }
          >
            <Text style={{ fontSize: 20, padding: 10 }}>
              {item.name}
            </Text>
          </TouchableOpacity>
        )}
      />
    </View>
  );
}
```

App.tsx

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import ListScreen from './ListScreen';
import DetailScreen from './DetailScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="List" component={ListScreen} />
        <Stack.Screen name="Detail" component={DetailScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Exercise 10 (1 mark)

Modify the code in Example 8 to include a price in the data object. Display the price in the detail screen. Show a screenshot of the output displayed on the detail screen after making this change.

Exercise 11 (1 mark)

What object does the Detail screen use to access the data passed from the previous screen?

The Detail screen uses `route.params` to access the passed data

Exercise 12 (1 mark)

What is the purpose of TouchableOpacity?

makes an element tappable and dims it briefly when pressed to give visual feedback.

Conclusion

I hope you can see how straightforward it is to set up a mobile application, whether using a native framework or a cross-platform framework like Flutter, Kotlin, or React Native. These frameworks are quite similar in many ways, and once you begin working on a real project, your skills will grow as you deepen your focus. This course is intended to give you a foundation and an understanding of how these tools work, so you can adapt and build whatever application is required.